

DisrtuCare — Software ↔ Hardware Communication

How the mobile app and the dispenser talk to each other

DisrtuCare — Technical Brief

June 2026

The big picture

One sentence

The phone app and the pill dispenser are **two HTTP peers on the same WiFi network**. The dispenser is a tiny web server; the app is its only client.



- **No Bluetooth, no native modules, no cloud.** Pure `fetch()` over the LAN.
- Works inside **Expo Go** because it only uses standard web APIs.
- The dispenser keeps running **autonomously** even with no app connected.

Why HTTP over WiFi

Design choices that shape the protocol

Decision	Consequence
HTTP/JSON on port 80	Debuggable from any browser; no SDK on the phone
Device = server, app = client	App discovers device by IP , not the reverse
Stateless REST + polling	No persistent socket to keep alive or reconnect
CORS enabled (*)	Works from web/Expo without proxy headaches
Schedule stored in EEPROM	Device survives reboots without the app

*The app never needs to be connected — it **configures** and **observes**. All time-critical logic (dispensing, confirmation) lives on the device.*

The API surface

Seven endpoints on the NodeMCU

Endpoint	Method	Purpose
/status	GET	Poll: schedule, time, and the latest event
/sync	POST	Push AM/PM times + medication name (→ EEPROM + LCD)
/settime	POST	Set the device clock from the phone
/led	POST	Turn the indicator LED on/off
/motor	POST	Rotate the dispenser by N slots (manual/test)
/diag	GET	Health check: I2C scan, RTC heap, WiFi RSSI

Connection lifecycle

What happens when you tap “Connect”

The app's `httpService.connect()` runs a fixed handshake:

1. **Resolve URL** — `setUrl("192.168.1.42")` → `http://192.168.1.42`
2. **Ping** GET `/status` with a 6 s timeout. No reply means “Disconnected”.
3. **Sync the clock** — POST `/settime` with the phone's current date & time (the device has no battery-backed RTC in this build).
4. **Start polling** — GET `/status` every **3 seconds**.
5. Emit connected; the IP is saved in SQLite for next launch.

Steps 2–3 mean: the moment the app connects, the device's clock is correct and its scheduled doses fire at the right local time.

Polling & the event model

How the device “pushes” without push

There is no socket — instead the device keeps a **monotonic counter**.

GET /status →

```
{  
  "am": "08:00", "pm": "20:00", "med": "Metformin",  
  "time": "14:30", "rtc": true, "awaiting": false,  
  "event_id": 3,  
  "last_event": { "evt": "confirmed", "type": "AM" }  
}
```

- Every hardware event increments **event_id** and stores **last_event**.
- The app remembers the last id it saw. When **event_id** **changes**, it fires the handler for **last_event** exactly once — no duplicates, no missed events.
- A failed poll makes the app flip to **Disconnected** automatically.

Event types

Three things the device can report

evt	type	Triggered by
dispensed	AM / PM	Scheduled time reached → motor rotated
dispensed	TEST	Manual POST /motor from the app
confirmed	AM / PM / TS	User pressed the physical button
error	—	Hardware fault (msg describes it)

- dispensed → app shows “pill released, awaiting confirmation”.
- confirmed → Dashboard **auto-logs the dose as Taken** and alerts the user.

Writing to the device

The POST payloads

Sync the schedule — POST /sync

```
{ "am": "08:00", "pm": "20:00", "med": "Metformin" }
```

Parsed into amHour/amMin/pmHour/pmMin, written to EEPROM, shown on the LCD.

Set the clock — POST /settime

```
{ "h": 14, "m": 30, "s": 0, "day": 16, "month": 6, "year": 2026 }
```

Manual actuators — POST /motor and POST /led

```
{ "slots": 1 }           { "state": 1 }
```

Every POST replies 200 {"ok":true}, or 400 {"ok":false,"error":"bad JSON"}.

End-to-end: a scheduled dose

From clock tick to “Taken” in the app

```
[device]  loop(): rtc time == amTime  →  dispense("AM")  
          └─ motorRotate(1)  (2048 half-steps ≈ ½ turn, one slot)  
              └─ LCD: "Dispensing" → "Done"  
                  └─ LED starts blinking, awaiting = true  
                      └─ pushEvent {evt:"dispensed", type:"AM"} (event_id++)
```

```
[app]      next /status poll sees new event_id  
           └─ shows "Dose released – please confirm"
```

```
[device]  user presses button → awaiting = false, LED solid 1.5 s  
          └─ pushEvent {evt:"confirmed", type:"AM"} (event_id++)
```

```
[app]      next poll  →  logs dose as TAKEN + alert
```

Robustness built into the protocol

Small details that keep it reliable

- **Timeouts everywhere** — 6 s `AbortSignal` on every request; a hung device never freezes the UI.
- **Multi-WiFi roaming** — the device knows several networks (`ESP8266WiFiMulti`) and rejoins the best available one; re-checks every ~8 s if dropped.
- **EEPROM persistence** — schedule + medication name survive power loss (magic byte `0xAB` validates the stored block).
- **Idempotent dispensing** — `lastDispensedMin` guard prevents a dose firing twice in the same minute; resets at midnight.
- **CORS preflight** — `OPTIONS` handled for every `POST` route.
- **Manual recovery** — `/diag` and `/selftest` let the app probe hardware without a full dispense cycle.

Summary

The communication model in five points

1. **Transport:** plain HTTP + JSON over the shared WiFi LAN — nothing else.
2. **Roles:** dispenser is the server (:80), the app is the only client.
3. **App → device:** REST writes (/sync, /settime, /led, /motor).
4. **Device → app:** a polled /status with an incrementing event_id that delivers dispensed / confirmed / error events.
5. **Autonomy:** the device dispenses and confirms on its own; the app just configures it and mirrors what happened.

Phone configures. Device acts. Status reconciles.